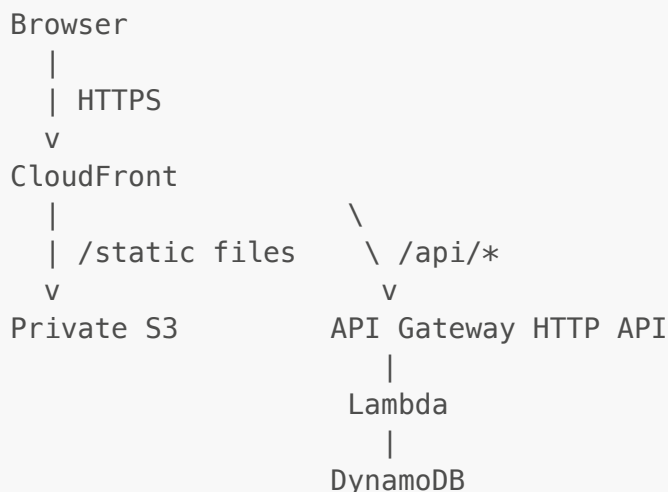


会議室予約カレンダー

会議室 31-205 / 31-202 / 31-204 / 31-309 向けの予約カレンダーです。AWS のサーバーレス構成で、HTTPS、非公開 S3、予約PIN、監視を含めて運用します。

構成



セキュリティ対策

このテンプレートには以下を含めています。

- S3 バケットは非公開
- CloudFront Origin Access Control で CloudFront だけが S3 を読める
- CloudFront で HTTPS にリダイレクト
- セキュリティヘッダーと CSP を付与
- API は CloudFront の同一オリジン `/api/*` 経由で利用
- API Gateway にスロットリングを設定
- 予約ごとに作成時の予約PINを設定し、そのPINで編集・削除
- Lambda の DynamoDB 権限は対象テーブルの `Scan / GetItem / PutItem / DeleteItem` に限定
- Lambda / API Gateway / DynamoDB の CloudWatch アラームを作成
- 任意でアラーム通知先メールアドレスを設定可能

必要なもの

- AWS アカウント
- AWS CLI
- AWS SAM CLI
- Node.js 20 以上

確認:

```
aws sts get-caller-identity
sam --version
node -v
npm -v
```

ローカル準備

```
cd reserve31_205
npm install
npm run check
```

ローカル確認では `data/reservations.json` に保存します。

```
npm start
```

```
http://localhost:3000
```

ローカルで管理用 PIN やハッシュ pepper を試す場合は `.env.local` を作成します。

```
DELETE_PIN=管理用PIN
DELETE_PIN_PEPPER=長いランダム文字列
```

AWS デプロイ

初回:

```
sam build
sam deploy --guided
```

`OriginVerifySecret` は長いランダム文字列を使います。

```
openssl rand -base64 32
```

入力例:

```
Stack Name: reserve31-205
AWS Region: ap-northeast-1
```

```
Parameter DeletePin: 管理用PIN (古い予約や緊急削除用)
Parameter OriginVerifySecret: ランダムな長い文字列
Parameter AlarmEmail: 通知先メールアドレス (不要なら空)
Parameter AllowedCorsOrigin: https://example.invalid
Confirm changes before deploy: Y
Allow SAM CLI IAM role creation: Y
Disable rollback: N
Save arguments to configuration file: Y
```

AlarmEmail を設定した場合、AWS から確認メールが届くので subscription を承認してください。

2 回目以降:

```
sam build
sam deploy --guided
```

今回のように **template.yaml** に新しい Parameters が追加された場合は、保存済み **samconfig.toml** だけでは足りないため **--guided** で再入力してください。

静的ファイルのアップロード

デプロイ後の Outputs から以下を控えます。

- **WebsiteUrl**: CloudFront の HTTPS URL
- **CloudFrontDistributionId**: CloudFront distribution ID
- **StaticAssetBucketName**: 非公開 S3 バケット名
- **ApiUrl**: API Gateway の直接 URL
- **ReservationsTableName**: DynamoDB テーブル名

CloudFront 経由では API を同一オリジンの **/api/reservations** として呼べるため、**config.js** は以下でよいです。

```
window.RESERVATION_API_BASE = "/api/reservations";
```

アップロード:

```
aws s3 cp index.html s3://YOUR_STATIC_ASSET_BUCKET_NAME/index.html --
content-type text/html
aws s3 cp styles.css s3://YOUR_STATIC_ASSET_BUCKET_NAME/styles.css --
content-type text/css
aws s3 cp app.js s3://YOUR_STATIC_ASSET_BUCKET_NAME/app.js --content-type
application/javascript
aws s3 cp config.js s3://YOUR_STATIC_ASSET_BUCKET_NAME/config.js --content-
type application/javascript
```

CloudFront キャッシュを消します。

```
aws cloudfront create-invalidation --distribution-id YOUR_DISTRIBUTION_ID --paths "/*"
```

ブラウザでは **WebsiteUrl** を開きます。

```
https://YOUR_DISTRIBUTION_DOMAIN.cloudfront.net
```

動作確認

CloudFront 経由:

```
curl https://YOUR_DISTRIBUTION_DOMAIN.cloudfront.net/api/health
```

期待値:

```
{"ok":true,"storage":"dynamodb"}
```

予約登録、重複予約の拒否、作成時に設定した予約PINによる編集・削除を確認します。カレンダー上の予約チップは、ダブルクリックすると編集画面を開けます。

API Gateway の直接 URL は、CloudFront が付ける秘密ヘッダーなしでは **403** になります。通常の利用では **WebsiteUrl** だけを使います。

現行システム同期

現行システム <https://www.r326.com/b/main.aspx?g=KQne6SJc286EYIb1a1> から、本日以降 90 日分の 31-309 予約を DynamoDB へ同期します。

- 同期元: りざぶ郎 31号館309会議室予約
- 同期先: このシステムの DynamoDB テーブル
- 同期時刻: 毎日 0:00 JST
- 同期予約の予約PIN: **1100**
- 同期予約の内部ID: **r326-<現行システムの予約ID>**

同期予約は **source: r326** と **externalId** を保存します。毎晩の同期では、本日以降 90 日分について現行システム側に存在する予約を作成・更新し、現行システム側から消えた **r326** 由来の予約だけを削除します。手入力した予約は削除対象にしません。

デプロイ後に一度だけ手動同期する場合:

```
FUNCTION_NAME=$(aws cloudformation describe-stack-resources \
  --stack-name reserve31-205 \
  --query "StackResources[?
LogicalResourceId=='ReservationsFunction'].PhysicalResourceId" \
  --output text)

aws lambda invoke \
  --function-name "$FUNCTION_NAME" \
  --cli-binary-format raw-in-base64-out \
  --payload '{"action":"sync-r326"}' \
  /tmp/r326-sync-result.json

cat /tmp/r326-sync-result.json
```

API 仕様

- GET /api/health: ヘルスチェック
- GET /api/reservations: 予約一覧取得
- POST /api/reservations: 新規予約登録
- PUT /api/reservations/:id: 予約変更
- DELETE /api/reservations/:id: 予約削除

変更・削除時は **X-Delete-Pin** ヘッダーに、予約作成時に設定した予約PINを指定します。

```
curl -X DELETE \
  -H "X-Delete-Pin: RESERVATION_PIN" \

https://YOUR_DISTRIBUTION_DOMAIN.cloudfront.net/api/reservations/YOUR_RESER
VATION_ID
```

POST /api/reservations の例:

```
{
  "room": "31-205",
  "startAt": "2026-06-12T10:00",
  "endAt": "2026-06-12T11:00",
  "purpose": "ゼミ打ち合わせ",
  "booker": "中澤",
  "deletePin": "1234"
}
```

会議場所

- 31-205
- 31-202
- 31-204

- 31-309

DynamoDB テーブル

`template.yaml` で以下を作成します。

- Partition key: `id` (String)
- Billing mode: `PAY_PER_REQUEST`
- SSE enabled

保存する属性:

- `id`
- `room`
- `startAt`
- `endAt`
- `purpose`
- `booker`
- `deletePinHash`
- `createdAt`
- `updatedAt`
- `source`
- `externalId`

重複チェックは、同じ会議場所の予約を Lambda 側で読み出して判定します。件数が大きく増える場合は、`room` と `startAt` を使った GSI 追加を検討してください。

バリデーション

- 必須項目が未入力の場合はエラー
- 会議場所が想定外の値の場合はエラー
- 日時形式が不正な場合はエラー
- 開始日時 $\geq$ 終了日時 の場合はエラー
- 同じ会議場所の重複時間帯は登録不可

ファイル構成

- `index.html`: 画面構造
- `styles.css`: スタイル定義
- `app.js`: フロントエンド描画と API 通信
- `config.example.js`: API URL 設定サンプル
- `server.js`: ローカル確認用 Express サーバー
- `api/lambda.js`: AWS Lambda ハンドラー
- `api/_lib/common.js`: 共通バリデーション
- `api/_lib/dynamodb-store.js`: DynamoDB アクセス
- `api/_lib/r326-sync.js`: 現行システム同期
- `template.yaml`: AWS SAM テンプレート

運用メモ

- AWS アクセスキーは root ユーザーで作らない
- 使わないアクセスキーは削除する
- `DELETE_PIN`, `DELETE_PIN_PEPPER`, `ORIGIN_VERIFY_SECRET`, `config.js` は Git 管理しない
- 月額事故防止のため AWS Budgets のアラートも別途設定する
- 独自ドメインを使う場合は CloudFront に ACM 証明書と Alternate domain name を追加する